

Nevera inteligente de medicamentos

Jeronimo Botero Quintero
Sara Muñoz Estrada
Luna Zapata Acuña
Electrónica digital II

Índice

1. identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	4
6. Explicación de funciones principales	5
7. Comunicación (si aplica)	6
8. Interfaz de usuario (si aplica)	7
9. Procedimiento de prueba	7
10. Manejo de errores y seguridad	7

1. identificación del proyecto

- **Nombre del proyecto:** Sistema avanzado de medición de reflejos y coordinación con ESP32 y MicroPython
- **Integrantes del equipo:** Jeronimo Botero Quintero, Sara Muñoz Estrada y Luna Zapata Acuña

2. Resumen

El proyecto consiste en el desarrollo de una nevera inteligente basada en ESP32, diseñada para monitorear las condiciones internas de almacenamiento mediante sensores de temperatura, humedad y detección de apertura de puerta. El sistema incorpora control de acceso mediante RFID, una pantalla OLED para visualización local y una interfaz web para el monitoreo remoto de las variables en tiempo real.

La información obtenida por los sensores es procesada mediante lógica difusa, la cual evalúa las condiciones de operación y clasifica el estado de la nevera en los niveles NORMAL, ALERTA o CRÍTICO. Dependiendo del nivel de riesgo detectado, el sistema activa indicadores visuales y sonoros para alertar al usuario.

Además, los datos son transmitidos por WiFi a un servidor local desarrollado en Python, donde se almacenan y visualizan a través de una página web. Este proyecto integra conceptos de sistemas embebidos, IoT y control inteligente para mejorar la supervisión de condiciones de almacenamiento.

3. Descripción del hardware

- **ESP32:** (uso de pines GPIO y manejo de registros para control de salidas)
- **Sensor de temperatura DS18B20:** pin 4
- **Sensor de humedad DHT22:** pin 15
- **Sensor magnético Reed Switch** pin 14
- **Servo motor (apertura y cierre de puerta):** pin 25
- **LED verde:** pin 26
- **LED rojo:** pin 27
- **Buzzer:** pin 32
- **Iluminación interna:** pin 22
- **Módulo relé:** pin 12

- **Pantalla OLED SSD1306 (I2C):** SDA: GPIO 21
SCL: GPIO 22.
- **Módulo RFID MFRC522 (SPI):** CS (SDA): pin 5
SCK: pin 18
MOSI: pin 23
MISO: pin 19

Diagrama de conexión:



4. Descripción del software

- **Lenguaje usado:** MicroPython, Python, HTML y JavaScript.

- **Librerías:** `machine`, `time`, `dht`, `network`, `urequests`, `onewire`, `ds18x20`, `ssd1306`, `mfr522`, `socketserve`, `http.server`, `json`, `threading`
- **IDE:** Thonny y visual studio code.

Flujo general del programa:

1. Inicialización de todos los sensores, actuadores y módulos de comunicación conectados a la ESP32.
2. Conexión automática a la red WiFi para permitir el envío de información al servidor.
3. Lectura periódica de la temperatura mediante el sensor DS18B20.
4. Lectura de la humedad relativa utilizando el sensor DHT22.
5. Detección del estado de la puerta mediante el sensor Reed Switch.
6. Control de acceso por RFID para validar usuarios autorizados.
7. Apertura o cierre de la puerta mediante el servomotor según la identificación recibida.
8. Procesamiento de las variables de temperatura, humedad y tiempo de apertura mediante lógica difusa.
9. Clasificación del estado de la nevera como NORMAL, ALERTA o CRÍTICO.
10. Activación de indicadores visuales y sonoros cuando se detectan condiciones de riesgo.
11. Visualización local de la información en la pantalla OLED.
12. Envío periódico de los datos al servidor web mediante comunicación WiFi.
13. Recepción y almacenamiento de los datos en el servidor desarrollado en Python.
14. Actualización automática de la interfaz web para mostrar las variables del sistema en tiempo real.
15. Registro de eventos e historial de monitoreo para consulta del usuario.

5. Estructura del código

- Archivo principal: `main.py`
- `servidor_web.py`
El proyecto se encuentra dividido en dos programas principales: uno ejecutado en la ESP32 para el monitoreo y control de la nevera, y otro ejecutado en el computador para la recepción y visualización de los datos mediante una página web.

- El código se organiza en diferentes secciones que permiten estructurar el funcionamiento del sistema:
 - Configuración de hardware: definición de pines para sensores (DS18B20, DHT22 y Reed Switch), módulo RFID, pantalla OLED, LEDs, buzzer, relé y servomotor.
 - Configuración de comunicaciones: conexión a la red WiFi y configuración de los servicios de comunicación con el servidor web y Telegram.
 - Definición de funciones: funciones encargadas de la lectura de sensores, control del servomotor, generación de señales sonoras, manejo de LEDs y actualización de la pantalla OLED.
 - Implementación de lógica difusa: definición de las funciones de pertenencia, cálculo del nivel de riesgo y clasificación del estado del sistema.
 - Gestión de acceso RFID: validación de usuarios autorizados y control de apertura o cierre de la puerta.
 - Bucle principal: ciclo infinito (while True) donde se realizan las lecturas de sensores, la evaluación del riesgo, la actualización de los indicadores y el envío de información al servidor.
- Código del servidor web

El código del servidor se encuentra organizado en las siguientes secciones:

 - Configuración inicial: definición de variables globales, almacenamiento de registros y configuración del servidor HTTP.
 - Interfaz web: estructura de la página utilizando HTML, CSS y JavaScript para la visualización de los datos.
 - Manejo de solicitudes: recepción de datos enviados por la ESP32 mediante peticiones HTTP POST y consulta de información mediante solicitudes GET.
 - Visualización de información: actualización de variables en tiempo real, historial de registros y representación gráfica de las funciones de pertenencia utilizadas en la lógica difusa.
 - Ejecución del servidor: puesta en marcha del servicio web y atención continua de las solicitudes realizadas por la ESP32 y los usuarios.

6. Explicación de funciones principales

- `leer_temperatura()`: : Obtiene la temperatura medida por el sensor DS18B20 y devuelve el valor en grados Celsius para su posterior procesamiento.
- `dht.measure()`: Realiza una nueva lectura del sensor DHT22 para actualizar los valores de humedad y temperatura ambiente.
- `abrir_puerta()`: Controla el servomotor para mover la puerta a la posición de apertura cuando se detecta un usuario autorizado.
- `cerrar_puerta()`: Regresa el servomotor a la posición de cierre y asegura nuevamente la puerta de la nevera.

- `beep_ok()`: Genera una señal sonora corta para indicar una operación correcta, como la identificación de una tarjeta autorizada.
- `beep_error()`: Activa una secuencia de sonidos para indicar acceso denegado o una condición de error.
- `beep_alarm()`: Genera una alarma sonora cuando el sistema detecta una condición crítica de funcionamiento.
- `actualizar_oled()`: Actualiza la información mostrada en la pantalla OLED, incluyendo temperatura, humedad, estado de la puerta y nivel de riesgo.
- `evaluar_riesgo_difuso()`: Procesa las variables de temperatura, humedad y tiempo de apertura de puerta para calcular el nivel de riesgo mediante lógica difusa.
- `clasificar_estado()`: Determina si el sistema se encuentra en estado NORMAL, ALERTA o CRÍTICO según el valor de riesgo calculado.
- `conectar_wifi()`: Establece la conexión de la ESP32 con la red inalámbrica y verifica su disponibilidad durante el funcionamiento.
- `enviar_datos_servidor()` *Envía periódicamente las variables monitoreadas al servidor web mediante solicitudes.*
- `procesar_rfid()`: Lee el identificador de las tarjetas RFID y verifica si corresponden a usuarios autorizados.
- `do_POST()`: Función del servidor web encargada de recibir y almacenar los datos enviados por la ESP32.
- `do_GET()`: Atiende las solicitudes realizadas desde el navegador y entrega la información necesaria para actualizar la página web.
- `actualizar()`: Función implementada en JavaScript que consulta periódicamente el servidor para actualizar en tiempo real los datos mostrados en la interfaz web.

7. Comunicación (si aplica)

Este proyecto utiliza conexión WiFi por medio del ESP32. Gracias a esto, los datos de temperatura, humedad, estado de la puerta y nivel de riesgo pueden enviarse a un servidor local para ser monitoreados. También se implementó un bot de Telegram que permite consultar el estado de la nevera y recibir alertas cuando ocurre alguna situación que pueda afectar la conservación de los medicamentos.

8. Interfaz de usuario (si aplica)

Se utilizaron diferentes formas de visualizar la informacion.

Se utilizó una pantalla OLED donde se pueden observar la temperatura, humedad y el estado de la puerta. Tambien se cuentan con leds que indican si la nevera esta funcionando o hay alguna falla, como tambien utilizamos buzzersque emiten sonidos frente a una falla.

9. Procedimiento de prueba

1. Conectar el ESP32 al computador.
2. Cargar el código en el ESP32.
3. Verificar que la pantalla OLED muestre los valores de temperatura y humedad.
4. Acercar una tarjeta RFID y comprobar que la puerta se abra o cierre correctamente.
5. Acercar una tarjeta no autorizada y verificar que en la pantalla muestre acceso denegado y active la alarma correspondiente.
6. Abrir la puerta durante varios segundos y observar cómo cambia el nivel de riesgo.
7. Simular cambios de temperatura o humedad para comprobar que se generen alertas cuando sea necesario.
8. Verificar que los datos sean enviados correctamente al servidor local.
9. Comprobar que las notificaciones lleguen al bot de Telegram cuando se presente una situación crítica.

10. Manejo de errores y seguridad

- Se intenta reconectar automáticamente al WiFi si se pierde la conexión.
- Solo las personas que tengan una tarjeta RFID registrada pueden abrir la nevera.
- Si ocurre un error al leer los sensores, el programa continúa funcionando sin detenerse.
- Si el servidor deja de responder, la nevera sigue realizando las mediciones y el monitoreo normalmente.
- Cuando se detecta una condición crítica, el sistema activa alarmas visuales, sonoras y puede enviar una notificación por Telegram.
- La lógica difusa ayuda a tomar decisiones teniendo en cuenta varias condiciones al mismo tiempo, evitando alertas innecesarias.
- Como mejora futura, se recomienda incluir una batería de respaldo para que el sistema siga funcionando en caso de un corte de energía.